PERFORMANCE INNOVATIVE TASK

A Greedy Solution to the Traveling Salesperson Problem

A five-landmark tour of Cagayan de Oro City

NEAREST NEIGHBOR | PYTHON DEMONSTRATION | 10–15 MINUTE VIDEO

Prepared for the TSP algorithm performance task | October 2026

The problem: visit every landmark, then return

Real-world scenario

A city visitor wants to document five recognizable locations in one tour, starting and finishing at Gaston Park.

TSP requirements

Visit each required landmark once, return to the starting location, and minimize the selected travel-distance measure.

Objective in this demonstration: minimize total modeled straight-line distance, in kilometers.

Five real locations in Cagayan de Oro

A	Gaston Park	8.475420° N, 124.642140° E
B	Plaza Divisoria	8.477750° N, 124.643778° E
C	Ayala Malls Centrio	8.485127° N, 124.651201° E
D	SM CDO Downtown	8.484383° N, 124.654322° E
E	Limketkai Mall	8.481970° N, 124.656570° E

A = starting and ending point. Coordinates are approximate published landmark centers.

Graph, objective and declared assumptions

Weighted graph G = (V, E)

Vertices: A, B, C, D, E
Edges: every pair of distinct landmarks
Weights: modeled km between center points
Graph type: complete, undirected, symmetric

What the measurements mean

We apply the Haversine formula to published latitude/longitude points. Distances are rounded to 0.001 km. They are NOT Google Maps road or walking distances.

No traffic, road barriers, travel times, entrance points, time windows, or one-way restrictions are modeled.

Input data: 5 × 5 distance matrix

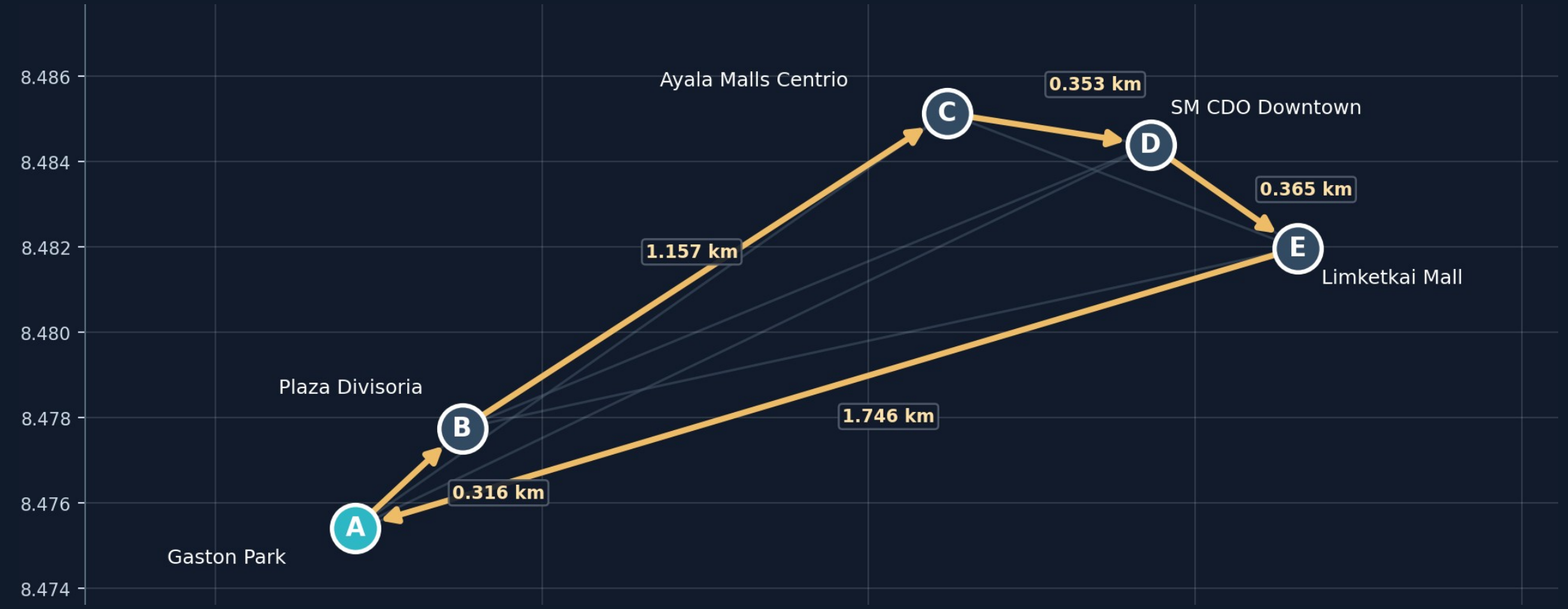
	A	B	C	D	E
A	0.000	0.316	1.469	1.670	1.746
B	0.316	0.000	1.157	1.374	1.483
C	1.469	1.157	0.000	0.353	0.687
D	1.670	1.374	0.353	0.000	0.365
E	1.746	1.483	0.687	0.365	0.000

How to read it

B → C = 1.157 km. The diagonal entries are zero. The table is symmetric because the graph is undirected.

Calculated Haversine straight-line weights (km); NOT Google Maps walking or driving distances.

Labeled weighted graph and Greedy tour



Gold = chosen edges with weights (km). All candidate weights are in the matrix. NOT a street map.

Assigned algorithm: Greedy / Nearest Neighbor

```
INPUT: vertices, pairwise distances, starting vertex
1   start at A; mark A visited
2   while an unvisited vertex remains:
3       choose the nearest UNVISITED vertex
4       move to it; mark it visited
5   return to A to close the tour
OUTPUT: complete tour and total distance
```

LOCALLY BEST NEXT STEP

HEURISTIC, NOT A GENERAL PROOF

Manual walkthrough: nearest eligible next move

FROM	CHOOSE	KM	WHY
A	B	**0.316**	0.316 < 1.469, 1.670, 1.746
B	C	**1.157**	1.157 < 1.374, 1.483
C	D	**0.353**	0.353 < 0.687
D	E	**0.365**	only E remains
E	A	**1.746**	return to the start

Route length = 0.316 + 1.157 + 0.353 + 0.365 + 1.746 = 3.937 km

How the Python implementation works

1 • Read data

Read TSP_Locations.csv and TSP_Distance_Matrix.csv; check IDs and symmetry.

2 • Run Greedy

From the current vertex, select the smallest distance among unvisited vertices.

3 • Validate

Close the tour, recalculate every leg, and confirm each vertex appears exactly once.

Main program uses only the Python standard library; no external packages or APIs are needed.

LIVE PYTHON EXECUTION | show the real run

Run from the folder containing all CSV files:

```
python TSP_Greedy_Implementation.py
```

Show these outputs live: input locations and matrix → greedy selections → closed tour → sum of weights → validation.

Do not replace this demonstration with screenshots of code/output. Show actual execution while recording.

Result: a valid 5-location closed tour

A → B → C → D → E → A

MODEL DISTANCE

3.937 km

TOUR VALIDATION

5 unique vertices visited
Return to A: confirmed

Recalculated from the exact 0.001-km values in the CSV distance matrix.

Does Greedy guarantee the shortest route?

In general: NO

Nearest Neighbor makes decisions based on the shortest next edge. Early choices can force a longer closing edge. Its repeated scans take $O(n^2)$ time.

For this dataset: YES, verified

An optional independent exhaustive audit checked all 4! = 24 fixed-start tours. The minimum was 3.937 km, matching our Greedy total.

The exhaustive audit is verification only: the assigned solver remains Greedy Nearest Neighbor.

Limitations, improvements and conclusion

Limitations

Straight-line distances are not real walking distances; accessibility and road geometry are ignored. Another dataset or starting point can yield a nonoptimal Greedy route.

Conclusion / next step

Our Greedy program creates a valid closed tour and achieves 3.937 modeled km. A future version could replace proxy weights with Google Maps walking/driving travel distances.

Answer the problem as modeled, not as an unrestricted claim about actual street travel.

References and map verification

Assigned algorithm source

“2.7 Greedy Algorithm,” The Mindanao Journal of Mathematics, Vol. 6 (2024), no. 2, pp. 79–111; excerpt at pp. 88–89.

Location / coordinate references

Gaston Park & Limketkai: Mapcarta / OpenStreetMap. Plaza Divisoria & Centrio: Wikipedia coordinates. SM Downtown: Wikidata.

Google Maps

OPEN ROUTE IN GOOGLE MAPS - WALKING PREVIEW >

Google Maps preview is for geographic context; numerical model weights come from coordinate calculations, not Maps directions.